



Παράλληλος και Κατανεμημένος Υπολογισμός I

Μοιραζόμενη μνήμη Συγχρονισμός σε συνθήκη

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2025



Συγχρονισμός σε συνθήκη

Ξανά συναγωνισμός δεδομένων

- Τώρα μας ενδιαφέρει **και η σειρά**, τι θα εκτελεστεί πρώτο. Θέλουμε να εκτελεστεί **πρώτα** η εγγραφή και **κατόπιν** η ανάγνωση (ή το if ή η ανάθεση..)

```
// shared variable  
my_data = 0
```

Λύνεται με συγχρονισμό σε συνθήκη.

```
main:  
    create/start ThreadA  
    create/start ThreadB
```

```
ThreadA :  
    ...  
    my_data = 1
```

```
    ...
```

```
ThreadB :  
    ...  
    print (my_data)  
    ή if (my_data == ) ...  
    ή local_var = my_data + ...
```

```
    ...
```

Συγχρονισμός σε συνθήκη

```
// shared variable  
my_data = 0
```

```
main:  
    create/start ThreadA  
    create/start ThreadB
```

```
ThreadA :  
    ...  
    my_data = 1  
    signal_condition  
    το A σηματοδοτεί και  
    συνεχίζει  
    ...
```

```
ThreadB :  
    ...  
    το B δεν μπορεί  
    να συνεχίσει  
    wait_condition  
    print (my_data)  
    ...
```

Τα `wait_condition`, `signal_condition` πρέπει να είναι ατομικές ενέργειες, δεν διακόπτονται, δεν επηρεάζονται από χρονοδιακοπή, αναστολή, γενικά από δρομολόγηση.

Παρακινδυνευμένη λύση με flag

```
// shared variable  
Volatile my_data = 0  
Volatile flag = False
```

```
main :  
    create/start ThreadA  
    create/start ThreadB
```

```
ThreadA :  
    ...  
    my_data = 1  
    flag = True
```

```
ThreadB :  
    ...  
    while (!flag) {}  
    print (my_data)
```



Volatile: άμεση ενημέρωση ιεραρχίας μνήμης.

Προβλήματα: ενεργός αναμονή, συναγωνισμός δεδομένων ανάγνωση/εγγραφή, και στο flag και στο my_data. Δεν είναι ατομική ενέργεια! Τι θα συμβεί αν έχουμε πολλά νήματα A,B?

Περιορισμένη λύση με αμοιβαίο αποκλεισμό

```
// shared variable
my_data = 0
flag = False
lock_type l
main :
    create/start ThreadA
    create/start ThreadB
```

ThreadA :

```
...
Lock(l)
my_data = 1
flag = True
Unlock(l)
```

...

Λόγω lock() - unlock() δεν χρειάζεται Volatile.

Πρόβλημα: επαναλαμβανόμενο lock() - unlock() στο ThreadB, ενεργός αναμονή με επιπλέον καθυστέρηση.

ThreadB :

...

```
while (true) {
    Lock(l)
    if (flag) {
        print (my_data)
        Unlock(l)
        break
    }
    Unlock(l)
} ...
```

Σηματοφόρος (Semaphore)

- Ένας σηματοφόρος είναι ένας ειδικός τύπος δεδομένων `sem_type` που λαμβάνει μη - αρνητικές ακέραιες τιμές.
- Μετά την αρχικοποίηση σε μια τιμή $s = N$, $N \geq 0$ ένα νήμα μπορεί να προσπελάσει έναν σηματοφόρο μόνο με δύο σύνθετες ατομικές ενέργειες:
- $P(s)$, $acquire(s)$, $down(s)$: προσπάθεια για είσοδο
- $V(s)$, $release(s)$, $up(s)$: έξοδος
- Λέγεται και *Counting Semaphore* (Σηματοφόρος Μετρητής) και προτάθηκε από το *Dijkstra* το 1968, εξ' ου και τα Ολλανδικά αρχικά P , V

$P(s)$: προσπάθεια για είσοδο

AN ($s == 0$)

TOTE αναστολή εκτέλεσης στο s

ΑΛΛΙΩΣ $s = s - 1$, [↑]είσοδος στο κρίσιμο τμήμα

μετά την αφύπνιση από την αναστολή

- Όλο το $P(s)$ αποτελεί ατομική ενέργεια. Μετά από την εκτέλεση του $P(s)$ είτε:

(α) το $s == 0$ και το νήμα / διεργασία που την εκτέλεσε αναστέλλεται στο s , ή

(β) το $s > 0$ και το νήμα / διεργασία εκτελεί $s = s - 1$ και εισέρχεται στο κρίσιμο τμήμα

- Προσοχή: μετά την αφύπνιση το νήμα δε ρωτά για την τιμή του σηματοφόρου αλλά εισέρχεται στο κρίσιμο τμήμα. Η ουρά έχει μνήμη.

$V(s)$: έξοδος

- Το νήμα που εξέρχεται
ΑΝ υπάρχουν νήματα που έχουν ανασταλεί στο s
ΤΟΤΕ τα αφυπνίζει, ώστε ένα να εισέλθει στο
κρίσιμο τμήμα
ΑΛΛΙΩΣ $s = s + 1$
- Όλο το $V(s)$ αποτελεί ατομική ενέργεια. Αν $s \geq N$ το
αποτέλεσμα εξαρτάται από την υλοποίηση.

Μηχανισμός ταυτοχρονισμού

Τα $P(s)$, $V(s)$ έχουν λειτουργία ανάλογη με τα $Lock()$, $Unlock()$, δηλαδή:

- Επιβάλλουν κανόνες σχετικής σειράς εκτέλεσης, δηλαδή ασθενή συνέπεια.
- Επιβάλλουν ενημέρωση των τιμών των μοιραζόμενων μεταβλητών που τροποποιήθηκαν.

Επιστροφή από αναστολή: δικαιοσύνη

- Όταν ένα νήμα εκτελέσει $P(s)$ σε σηματοφόρο s και αποτύχει τότε η εκτέλεσή του αναστέλλεται, και τοποθετείται σε ουρά που υποδεικνύει ότι έχει ανασταλεί στον συγκεκριμένο σηματοφόρο s .
- Όταν ένα άλλο νήμα εκτελέσει $V(s)$ στον ίδιο σηματοφόρο s , τότε, ανάλογα με την υλοποίηση αφυπνίζεται:
 - (α) κάποιο από τα νήματα της ουράς
 - (β) το νήμα στην κεφαλή της ουράς (FIFO)
- Ο σηματοφόρος μπορεί να είναι δίκαιος (ισχυρός) ή όχι, ανάλογα με την υλοποίηση.

Συγχρονισμός σε συνθήκη με σηματοφόρο 1

- Οι λειτουργίες $P()$ και $V()$ μπορεί να εκτελεστούν από διαφορετικά νήματα. Είναι ατομικές.

Σε αυτή την περίπτωση:

- Το $P(s)$ ισοδυναμεί με `wait`
- Το $V(s)$ ισοδυναμεί με `signal`
- Ο σηματοφόρος έχει ουρά με μνήμη: το $V()$ μπορεί να προηγηθεί του $P()$.

Συγχρονισμός σε συνθήκη με σηματοφόρο 2

```
// shared variable  
sem_type my_sem(0)
```

```
main:  
    create/start ThreadA  
    create/start ThreadB
```

ThreadA :

```
...  
my_data = 1  
V(my_sem)
```

το A ή κάνει το my_sem = 1
ή σηματοδοτεί το B και
συνεχίζει

```
...
```

ThreadB :

```
...  
το B δεν μπορεί να  
συνεχίσει γιατί my_sem == 0
```

```
P(my_sem)  
print (my_data)
```

το B κάνει το my_sem = 0

```
...
```

Συγχρονισμός σε συνθήκη με σηματοφόρο 3

- Αν το νήμα B εκτελέσει πρώτο το $P()$:
το $my_sem == 0$ επομένως το νήμα B θα ανασταλεί και θα προχωρήσει μόνο όταν το νήμα A προχωρήσει για να εκτελέσει το $V()$.
- Αν το νήμα A εκτελέσει πρώτο το $V()$:
το $my_sem == 1$ επομένως το νήμα B όταν εκτελέσει το $P()$ θα προχωρήσει κάνοντας το $my_sem = 0$.

Σηματοφόρος: μια λύση για δύο προβλήματα

- Αρχικά $s = 1$.
 - Δύο νήματα A , B εκτελούν και τα δύο $P(s) \dots V(s)$
 - Αμοιβαίος αποκλεισμός: ένα νήμα από τα A , B προχωρεί το άλλο περιμένει (δεν ενδιαφέρει η σειρά).
-
- Αρχικά $s = 0$.
 - Δύο νήματα A , B . Το νήμα A εκτελεί $V(s)$ και το νήμα B εκτελεί $P(s)$.
 - Συγχρονισμός σε συνθήκη: το νήμα B περιμένει τη σηματοδοσία του νήματος A (η σειρά έχει σημασία).

Σηματοφόρος: πολλαπλή σημασιολογία

- Ο μεταγλωττιστής δεν μπορεί να ελέγξει με βάση τη θέση των $P()$, $V()$ μέσα στον κώδικα.
- Η τιμή του s μπορεί να είναι οποιαδήποτε ≥ 0 .
- Δεν υπάρχει εύκολη έκφραση λογικών συνθηκών, μόνο αριθμητικών, με αποτέλεσμα μερικές φορές δυσνόητο κώδικα.
- Ο προγραμματιστής μπορεί να κάνει πολύ έξυπνα πράγματα αλλά και πολύ δυσνόητα λάθη! Δύσκολο debugging.

Μεταβλητές συνθήκης (condition variables)

- Οι μεταβλητές συνθήκης χρησιμοποιούνται σε συνδυασμό με τα κλειδώματα για να πετύχουμε συγχρονισμό σε συνθήκη.
- Είναι λιγότερο πυκνές από τους σηματοφόρους αλλά έχουν:
- Συντακτικό που μπορεί να ελέγξει ο μεταγλωττιστής
- Πιο περιφραστική σημασιολογία, ευελιξία στην έκφραση αριθμητικών και λογικών συνθηκών.
- Ο προγραμματιστής ελέγχεται από τον μεταγλωττιστή και έτσι είναι αναγκαστικά πειθαρχημένος.

Μεταβλητές συνθήκης 1

- Οι μεταβλητές συνθήκης είναι ένας ειδικός τύπος μεταβλητής, στην ουσία ένα σύνολο (όχι απαραίτητα ουρά FIFO) –δεν είναι η λογική συνθήκη καθ' αυτή!
- Μια μεταβλητή συνθήκης V ορίζεται σε συνδυασμό με ένα κλείδωμα L .
- Μια μεταβλητή συνθήκης V διαθέτει ένα σύνολο αναμονής.
- Η τιμή της μεταβλητής συνθήκης V ισούται με το πλήθος των νημάτων στο σύνολο αναμονής.

Μεταβλητές συνθήκης 2

- Ένα νήμα μπορεί να εκτελέσει σε μια μεταβλητή συνθήκης δύο βασικές λειτουργίες:
- Λειτουργία αναμονής, `await (L, V)`:
ένα νήμα ελευθερώνει το κλείδωμα `L` και αναστέλλεται στην ουρά της μεταβλητής συνθήκης `V`.
- Λειτουργία σηματοδότησης, `signal (V)`, `signal_all(V)`, `broadcast()`:
ένα νήμα ενημερώνει τη μεταβλητή συνθήκης `V` και αφυπνίζει ένα ή περισσότερα νήματα από το σύνολο αναμονής της `V`.

Μεταβλητές συνθήκης 3

- Ένα νήμα A καταλαμβάνει ένα κλείδωμα L αλλά για να προχωρήσει στην εκτέλεση πρέπει να ελέγξει αν ικανοποιείται μια συνθήκη συγχρονισμού C.
- Αν η συνθήκη συγχρονισμού C δεν ικανοποιείται τότε το νήμα ελευθερώνει το κλείδωμα L και αναστέλλει την εκτέλεσή του: αναμένει στο σύνολο αναμονής της μεταβλητής συνθήκης V: `await (L, V)`.
- Ένα άλλο νήμα B μπορεί τότε να καταλάβει το κλείδωμα L και να προχωρήσει την εκτέλεσή του κώδικά του, ώστε να τροποποιήσει τη συνθήκη C, και να ενημερώσει τη μεταβλητή συνθήκης V: `signal (V)`, `signal_all(V)` ή `broadcast()`.

Μεταβλητές συνθήκης 4

- Τα νήματα που αφυπνίστηκαν, ερωτούν για την ικανοποίηση της συνθήκης.
- Αν η συνθήκη ικανοποιείται, αυτά τα νήματα αλλά και άλλα που πιθανά βρίσκονται εκτός κρίσιμου τμήματος, προσπαθούν να εισέλθουν, και ανταγωνίζονται για το κλείδωμα L .
- Ένα νήμα το καταλαμβάνει και τα υπόλοιπα αναστέλλουν την εκτέλεσή τους.
- Αν η συνθήκη δεν ικανοποιείται τα νήματα πάλι αναστέλλουν την εκτέλεσή τους και εισέρχονται στο σύνολο αναμονής.

Μεταβλητές συνθήκης 5

- Οι μεταβλητές συνθήκης δεν έχουν μνήμη. Αν η λειτουργία `signal(V)` ή `signal_all(V)` ή `broadcast(V)` συμβεί σε άδειο σύνολο αναμονής τότε δεν καταγράφεται. Το `await(L, V)` πρέπει να προηγείται του `signal(V)`.
- Η ερώτηση για την ικανοποίηση της συνθήκης διασφαλίζει ότι το `signal(V)` θα προλάβει τη `await(L, V)`

lock(L)

while (!C) await(L, V) ...

Μεταβλητές συνθήκης 6

- Συνήθης δομή κώδικα για επίτευξη συγχρονισμού με κλείδωμα και μεταβλητή συνθήκης:

```
lock(L)  
while (!C) await(L, V)
```

```
... modify C in critical section
```

```
signal(V) // signal_all(V) ή broadcast(V)  
unlock(L)
```

Δύο λύσεις για δύο προβλήματα !

- Αν έχουμε **μόνο** κλειδώματα : αμοιβαίος αποκλεισμός
- Αν έχουμε κλειδώματα **και** μεταβλητές συνθήκης: συγχρονισμός σε συνθήκη

lock(L)

while (!C) await(L, V)

... modify C in critical section

signal(V) // signal_all(V) ή broadcast(V)

unlock(L)

Συγχρονισμός σε συνθήκη με κλείδωμα και μεταβλητή συνθήκης 1

```
// shared variable  
my_data = 0
```

```
flag = False           //μεταβλητή (λογική συνθήκη)  
lock_type my_lock     //κλείδωμα  
cond_type my_cond     //μεταβλητή συνθήκης
```

main:

```
create/start ThreadA  
create/start ThreadB
```

ThreadA :

```
...  
Lock(my_lock)  
my_data = 1  
flag = True  
signal(my_cond)  
Unlock(my_lock)
```

...
το A θέτει flag = 1,
σηματοδοτεί και
συνεχίζει

ThreadB :

```
...  
το B δεν μπορεί να  
συνεχίσει γιατί flag = 0  
και δεν έχει λάβει σήμα  
Lock(my_lock)  
while (!flag)  
    await(my_lock, my_cond)  
print (my_data)  
flag = False  
Unlock(my_lock)  
...
```

Συγχρονισμός σε συνθήκη με κλείδωμα και μεταβλητή συνθήκης 2

- Αν το νήμα B καταλάβει πρώτο το `my_lock`:
το `flag == False` επομένως το νήμα B θα ελευθερώσει το `my_lock` και θα εκτελέσει το `while..await()`.
Όταν το νήμα A καταλάβει το `my_lock` εκτελεί `flag = True` και `signal()` --που αφυπνίζει το νήμα B από το `await()` -- και τελικά ελευθερώνει το `my_lock`.
Όταν το νήμα B καταλάβει το `my_lock` βρίσκει `flag == True` δεν μπαίνει στο `while..await()`, συνεχίζει και τελικά ελευθερώνει το `my_lock`.

Συγχρονισμός σε συνθήκη με κλείδωμα και μεταβλητή συνθήκης 3

- Αν το νήμα A καταλάβει πρώτο το `my_lock`:

θέτει το `flag = True`, εκτελεί `signal()` --που χάνεται γιατί το νήμα B δεν έχει μπει στο `while..await()` -- και τελικά ελευθερώνει το `my_lock`.

Το νήμα B όταν καταλάβει το `my_lock` βρίσκει `flag == True` δεν μπαίνει στο `while..await()`, συνεχίζει και τελικά ελευθερώνει το `my_lock`.

Signal and continue / Signal and wait

- Signal and continue: το νήμα που στέλνει το σήμα ενημέρωσης συνεχίζει και ελευθερώνει το κλείδωμα, ώστε να μην εμπλακεί στον ανταγωνισμό της ανακατάληψης του κλειδώματος.

**signal(V) ή signal_all(V) ή broadcast()
unlock(L)**

- Signal and wait: σε άλλες υλοποιήσεις το νήμα που στέλνει το σήμα ενημέρωσης αναστέλλει την εκτέλεσή του στη μεταβλητή συνθήκης V (λιγότερο συνηθισμένο, συζήτηση παρακάτω).



Επόπτες (monitors)

Επόπτες (monitors) 1

- Οι επόπτες (ή ελεγκτές ή επιτηρητές) είναι ένας αντικειμενοστρεφής μηχανισμός ταυτοχρονισμού υψηλού επιπέδου.
- Συνθέτει τα μοιραζόμενα δεδομένα, το κλείδωμα και τις μεταβλητές συνθήκης σε ένα αντικείμενο.
- Ενσωματώνει διαφανώς τις λειτουργίες
lock- unlock και await- signal/signal_all
- Οι επόπτες είναι παθητικά αντικείμενα: ελέγχουν την πρόσβαση νημάτων και προστατεύουν τα μοιραζόμενα δεδομένα αλλά δεν εκτελούν κώδικα, όπως τα νήματα.

Επόπτες 2

Οι επόπτες έχουν τα εξής συστατικά:

- Ένα σύνολο από ιδιωτικές μοιραζόμενες μεταβλητές: η κατάσταση (state) του επόπτη
- Ένα σύνολο από αμοιβαία αποκλειόμενες διαδικασίες επόπτη που παρέχουν δημόσια διεπαφή στα μοιραζόμενα δεδομένα.
- Ένα σύνολο από ιδιωτικές μεταβλητές συνθήκης που υλοποιούν συγχρονισμό σε συνθήκες αν απαιτείται.
- Κώδικα αρχικοποίησης, και πιθανώς άλλες διαδικασίες, που όμως δεν τροποποιούν τα μοιραζόμενα δεδομένα ή είναι ιδιωτικές.

Διαδικασίες επόπτη

- Μόνο οι υπογραφές των διαδικασιών είναι ορατές εκτός επόπτη.
- Ο μόνος τρόπος για ένα νήμα / διεργασία να προσπελάσει μοιραζόμενες μεταβλητές επόπτη είναι να καλέσει μια διαδικασία επόπτη.
- Οι μεταβλητές του επόπτη είναι μοιραζόμενες για όλες τις διαδικασίες του επόπτη.
- Οι διαδικασίες επόπτη μπορεί να έχουν και τοπικές μεταβλητές.
- Οι διαδικασίες επόπτη και ο κώδικας αρχικοποίησης του επόπτη δεν μπορούν να προσπελάσουν μεταβλητές εκτός του επόπτη εκτός αν πρόκειται για παραμέτρους των διαδικασιών.

Αμοιβαίος αποκλεισμός με επόπτη

- Ο αμοιβαίος αποκλεισμός είναι έμμεσος: σε κάθε χρονική στιγμή εκτελείται μόνο μια διαδικασία επόπτη.
- Κάθε επόπτης διαθέτει ένα κλείδωμα, που αφορά το σύνολο του επόπτη, και μια ουρά εισόδου νημάτων / διαδικασιών.
- Αν ένα νήμα / διεργασία A εκτελεί μια διαδικασία επόπτη και ένα άλλο νήμα / διεργασία B προσπαθήσει να εκτελέσει μια διαδικασία επόπτη (όχι μόνο την ίδια με αυτή που εκτελείται αλλά και οποιαδήποτε άλλη διαδικασία επόπτη) τότε το νήμα / διεργασία B τίθεται σε αναστολή και τοποθετείται στην ουρά εισόδου του συγκεκριμένου επόπτη.

Κρίσιμο τμήμα σε επόπτη

- Αφού οι μοιραζόμενες μεταβλητές του επόπτη είναι προσπελάσιμες μόνο μέσω των διαδικασιών επόπτη, και..
- τα νήματα / διεργασίες μπορούν να προσπελάσουν τις μεταβλητές του επόπτη μόνο μέσω των διαδικασιών επόπτη, και..
- μόνο μια διαδικασία επόπτη εκτελείται σε κάθε χρονική στιγμή, συνεπάγεται ότι..
- κάθε προσπάθεια (και τροποποίηση) των μεταβλητών επόπτη συμβαίνει σε καθεστώς αμοιβαίου αποκλεισμού, δηλαδή το σύνολο του κώδικα των διαδικασιών επόπτη αντιμετωπίζεται ως ένα κρίσιμο τμήμα.

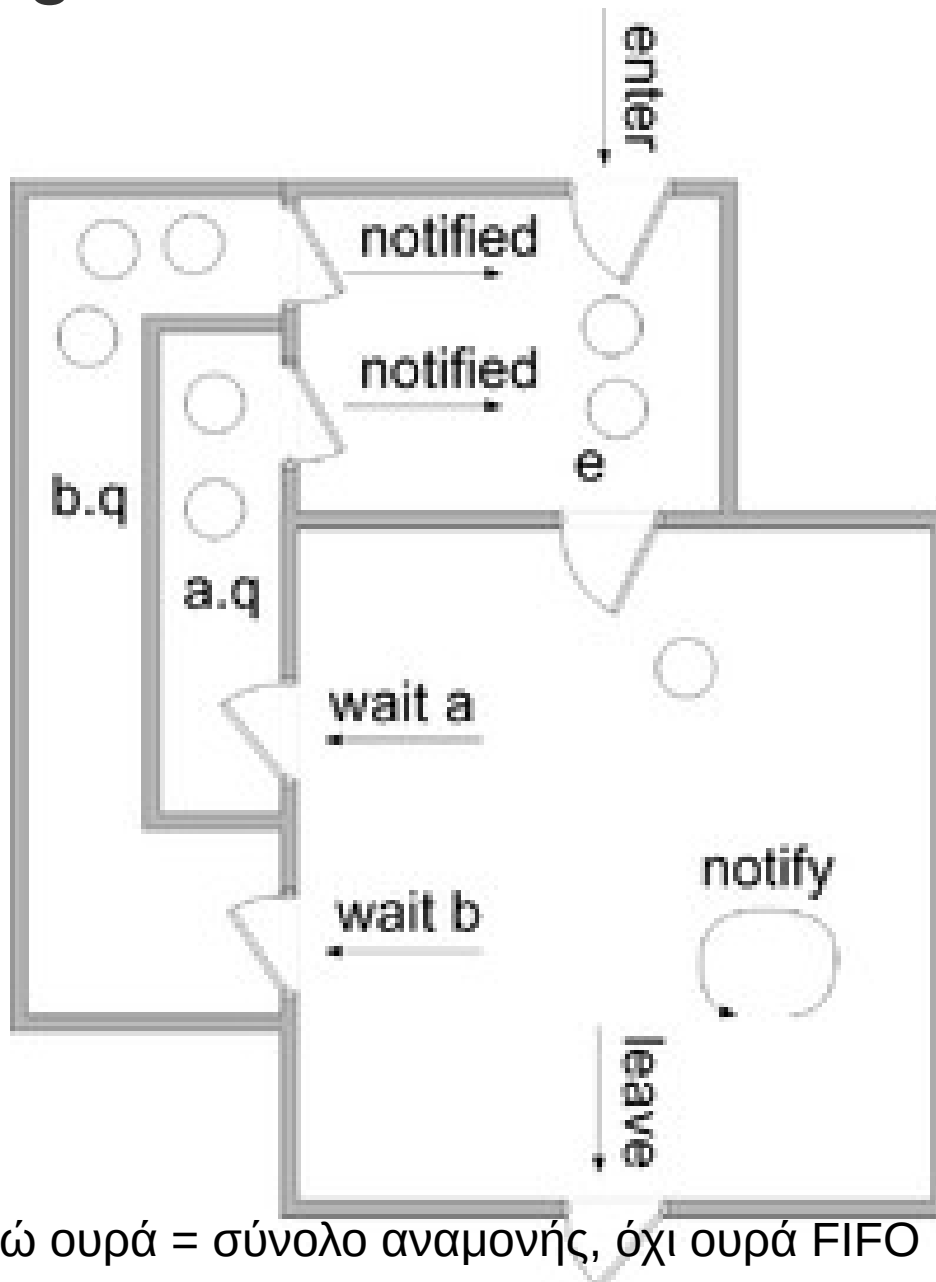
Συγχρονισμός σε συνθήκη με επόπτη 1

- Ο συγχρονισμός σε συνθήκη είναι άμεσος: πρέπει να προγραμματιστεί με τη χρήση συνθηκών συγχρονισμού και μεταβλητών συνθήκης του επόπτη.
- Οι συνθήκες συγχρονισμού αφορούν ιδιωτικές μεταβλητές του επόπτη και είναι προσπελάσιμες μόνο μέσα από τις διαδικασίες του επόπτη.
- Οι μεταβλητές συνθήκης του επόπτη είναι επίσης προσπελάσιμες μόνο μέσα από τις διαδικασίες του επόπτη.

Συγχρονισμός σε συνθήκη με επόπτη 2

- Κάθε μεταβλητή συνθήκης V διαθέτει τη δική της ουρά αναμονής και είναι προσπελάσιμη μέσω των δύο βασικών λειτουργιών:
 - λειτουργία αναμονής: $\text{wait}(V)$
 - λειτουργία ενημέρωσης: $\text{notify}(V)$
- Οι μεταβλητές συνθήκης του επόπτη είναι εξ' ορισμού συνδεδεμένες με το (έμμεσο) κλείδωμα του επόπτη, επομένως δεν απαιτείται σχετικός καθορισμός.

Ουρές* επόπτη signal and continue



- *e*: ουρά εισόδου επόπτη
- *a.q*, *b.q*: ουρές αναμονής (*wait*) στις μεταβλητές συνθήκης *a*, *b*
- Το ενεργό νήμα εκτελεί *notify()* ή *notify_all()* και συνεχίζει προς την έξοδο.
- Τα νήματα που αφυπνίζονται μετακινούνται (FIFO) στο τέλος της ουράς *e*.
- Το νήμα στη κεφαλή της ουράς *e* καταλαμβάνει τον επόπτη.
- Επειδή μεταξύ αφύπνισης και εκτέλεσης μεσολαβεί πιθανή αναμονή είναι **απαραίτητο το *while* στο *wait***.

* Εδώ ουρά = σύνολο αναμονής, όχι ουρά FIFO

Αμοιβαίος αποκλεισμός με επόπτη 1

```
//class definition
monitor_class my_monitor
    private data

    private init_procedure (d)
        data = d
        flag = False

    monitor_procedure p()
        print (data)

    monitor_procedure q()
        data = 1
```

Αμοιβαίος αποκλεισμός με επόπτη 2

```
// shared variable
```

```
a_monitor = new my_monitor(0)
```

```
main:
```

```
    create/start ThreadA
```

```
    create/start ThreadB
```

```
ThreadA :
```

```
    ...
```

```
a_monitor.q();
```

```
ThreadB :
```

```
    ...
```

το B δεν μπορεί να
συνεχίσει γιατί εκτελείται
η monitor procedure p()

```
a_monitor.p();
```

Συγχρονισμός σε συνθήκη με επόπτη 1

```
//global class definition
```

```
monitor_class my_monitor
```

```
    private data
```

```
    private flag
```

```
    private cond_type cv
```

```
    private init_procedure (d)
```

```
        data = d
```

```
        flag = False
```

```
    monitor_procedure w()
```

```
        while (!flag) wait(cv)
```

```
        print (data)
```

```
        flag = False
```

```
        notify(cv)
```

```
// μπορεί να λείπει εδώ
```

```
    monitor_procedure s()
```

```
        while (flag) wait(cv)
```

```
        data = 1
```

```
        flag = True
```

```
        notify(cv)
```

```
// μπορεί να λείπει εδώ
```


Συγχρονισμός σε συνθήκη με επόπτη 2

```
// shared variable
```

```
a_monitor = new my_monitor(0)
```

```
main:
```

```
    create/start ThreadA
```

```
    create/start ThreadB
```

```
ThreadA :
```

```
    ...
```

```
a_monitor.s();
```

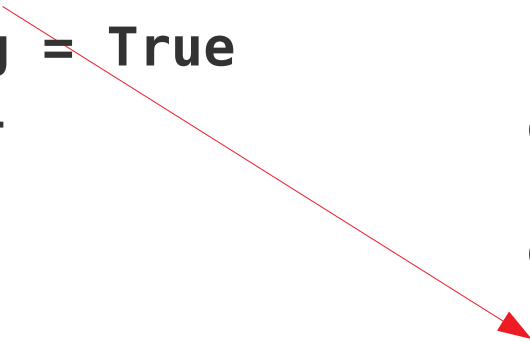
το A θέτει flag = True
σηματοδοτεί και
συνεχίζει

```
ThreadB :
```

```
    ...
```

το B δεν μπορεί να
συνεχίσει γιατί flag =
False και δεν έχει λάβει
σήμα

```
a_monitor.w();
```



Συγχρονισμός σε συνθήκη με επόπτη 3

- Η σημασιολογία της εκτέλεσης είναι όμοια με αυτή των μεταβλητών συνθήκης, μόνο που ο αμοιβαίος αποκλεισμός υλοποιείται από τις διαδικασίες επόπτη.
- Τα `περιττά` wait-notify() χρησιμοποιούνται για ομοιομορφία-συνέπεια στον κώδικα.

Σύνοψη εποπτών

- Οι επόπτες δεν είναι ξεχωριστός μηχανισμός ταυτοχρονισμού αλλά αντικειμενοστρεφής υλοποίηση κλειδωμάτων και μεταβλητών συνθήκης.
- Ο χειρισμός του αμοιβαίου αποκλεισμού δεν επιτρέπει τις λεπτομέρειες των κλειδωμάτων, δηλαδή έχουν περιορισμένη διακριτότητα και ευελιξία.
- Όμως είναι εύχρηστοι, συγκεντρώνουν τους μηχανισμούς ταυτοχρονισμού στα δεδομένα και διευκολύνουν στην απόκρυψη πληροφορίας.



Συγχρονισμός σε συνθήκη στη Java

Συγχρονισμός σε συνθήκη στη Java

- Στη Java, οι μεταβλητές συνθήκης, όπως και το κλασσικό κλείδωμα, δεν είναι ενσωματωμένο στη κύρια γλώσσα αλλά παρέχονται στο πακέτο `java.util.concurrent`
- Οι σχεδιαστές της Java έχουν επιλέξει ως κύριο μηχανισμό ταυτοχρονισμού τον συνδυασμό `synchronized` με `wait-notify`, επειδή είναι υψηλότερου επιπέδου και αντικειμενοστρεφής. Μοιάζει πολύ με τους επόπτες.
- Για λόγους γενικότητας και φορητότητας θα ξεκινήσουμε με τις μεταβλητές συνθήκης, που είναι διαθέσιμες σε όλες σχεδόν τις γλώσσες.

Μεταβλητές συνθήκης στη Java 1

- Στη Java οι μεταβλητές συνθήκης Condition, βρίσκονται στο πακέτο `java.util.concurrent`.
- Είναι αντικείμενα που ορίζονται πάντα σε συνδυασμό με ένα κλείδωμα Lock (ReentrantLock, ReadWriteLock, ReentrantReadWriteLock), πχ:

```
Lock lock = new ReentrantLock();  
Condition cond = lock.newCondition();
```

- Βασικές μέθοδοι: `await()`, `signal()` ή `signalAll()`
- Παρέχονται επιπλέον μέθοδοι δίνουν ευελιξία στη χρήση, δεν καλύπτονται εδώ.

Αντικείμενα Java: περίπου επόπτες

Κάθε αντικείμενο είναι εν δυνάμει επόπτης:

- Διαθέτει ένα εγγενές κλείδωμα ή κλείδωμα επόπτη.
- Διαθέτει μια εγγενή μεταβλητή συνθήκης

Προσοχή:

- τα νήματα είναι αντικείμενα !

Επόπτες στη Java: διαδικασίες επόπτη 1

- Οι μέθοδοι ενός αντικειμένου μπορεί να είναι διαδικασίες επόπτη (να ελέγχονται από τον επόπτη) ή όχι.
- Μπορεί δηλαδή στο ίδιο αντικείμενο – επόπτη να συνυπάρχουν μέθοδοι επόπτη και απλές μέθοδοι.
- Οι μέθοδοι επόπτη ξεχωρίζουν με το προσδιοριστικό `synchronized` πριν το όνομα της μεθόδου.
- Οι απλές μέθοδοι δεν υπόκεινται στους περιορισμούς του επόπτη.

Επόπτες στη Java: διαδικασίες επόπτη 2

```
class myClass {  
    ...  
    synchronized aMonitorMethod {  
        ...  
    }  
    synchronized bMonitorMethod {  
        ...  
    }  
    simpleMethod {  
        ...  
    }  
}
```

Επόπτες στη Java: τμήματα μεθόδων 1

- Στη Java το κρίσιμο τμήμα δεν είναι απαραίτητα ολόκληρη μέθοδος επόπτη αλλά μπορεί να είναι και τμήμα μιας απλής μεθόδου του επόπτη, το οποίο λέγεται `synchronized block` και δηλώνεται ως

...

```
synchronized (object reference) {
```

...

```
}
```

όπου το `object reference` πρέπει να είναι μια έκφραση που επιστρέφει αναφορά σε ένα αντικείμενο – συνήθως η έκφραση είναι το **this**.

Επόπτες στη Java: τμήματα μεθόδων 2

```
class myClass {  
    ...  
    aMonitorMethod {  
        ... // non critical section  
        synchronized (this) {  
            ... // critical section  
        }  
    }  
  
    synchronized bMonitorMethod {  
        ...  
    }  
}
```

Επόπτες στη Java: μεταβλητές συνθήκης 1

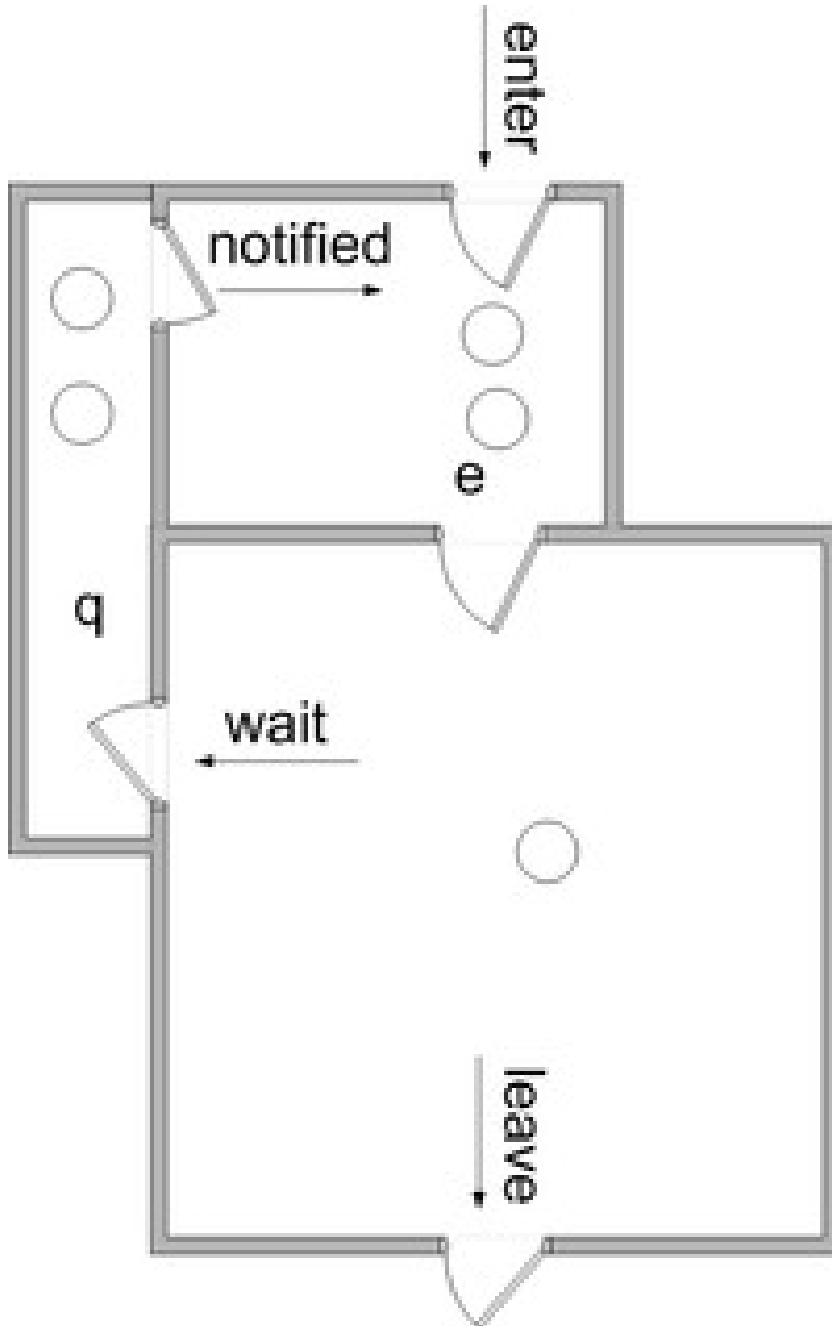
- Υπάρχει μόνο μια μεταβλητή συνθήκης και γι' αυτό δεν δηλώνεται ρητά.
- Επομένως υπάρχει μόνο μια ουρά αναμονής στη μεταβλητή συνθήκης σε κάθε αντικείμενο – επόπτη, το λεγόμενο wait set.
- Η ουρά αυτή είναι εγγενής όπως και το κλείδωμα επόπτη (δεν απαιτείται ορισμός της ούτε υπάρχει δυνατότητα διαχείρισης από το πρόγραμμα).
- Το εγχειρίδιο της JVM δεν διευκρινίζει αν υλοποιείται ως ουρά FIFO ή ως σύνολο (set).

Επόπτες στη Java: μεταβλητές συνθήκης 2

- Η προσπέλαση στις μεταβλητές συνθήκης γίνεται με τις μεθόδους :

`wait()`, `notify()`, `notifyAll()`

- Η Java για την υλοποίηση των `notify()`, `notifyAll()` χρησιμοποιεί τη μέθοδο `signal and continue`.



Επόπτης στη Java

- e: ουρά εισόδου επόπτη
- q: 'ουρά' αναμονής (wait) σε όλες τις συνθήκες
- Το ενεργό νήμα εκτελεί notify() ή notify_all() και συνεχίζει προς την έξοδο.
- Τα νήματα που αφυπνίζονται μετακινούνται (όχι απαραίτητα FIFO) στο τέλος της ουράς e.
- Το νήμα στη κεφαλή της ουράς e καταλαμβάνει τον επόπτη

Επόπτες στη Java: wait 1

- Η μέθοδος `wait()` καλείται μόνο μέσα από `synchronized` μέθοδο ή `synchronized block`.
- Το νήμα που την καλεί, ελευθερώνει το κλείδωμα του επόπτη και αναστέλλεται (μεταφέρεται στο `wait set`) σε αναμονή σήματος `notify()`.
- Μετά την αφύπνιση το νήμα πρέπει να ανακαταλάβει το κλείδωμα του επόπτη πριν συνεχίσει από την επόμενη γραμμή κώδικα μετά τη κλήση `wait ()`.
- Για να υπάρξει εκ νέου έλεγχος της συνθήκης συγχρονισμού η κλήση `wait()` βρίσκεται μέσα σε βρόχο `while`.

Επόπτες στη Java: wait 2

- Οι μέθοδοι
wait(long millis) και wait(long millis, int nanos)
επιτρέπουν την χρονοδιακοπή της αναμονής.
- Εφ' όσον υπάρχει αναστολή εκτέλεσης η κλήση
περικλείεται σε try-catch

```
try {  
    while (someTest) wait();  
} catch (InterruptedException e) {}
```


Επόπτες στη Java: notify

- Οι μέθοδοι `notify()`, `notifyAll()` καλούνται μόνο μέσα από `synchronized` μέθοδο ή `synchronized block`.
- Το νήμα που την καλεί, αφυπνίζει ένα (`notify`) ή όλα (`notifyAll`) τα νήματα που αναμένουν στο `wait set` του επόπτη. Αν δεν υπάρχουν νήματα το σήμα αφύπνισης χάνεται.
- Η κλήση `notify` συνήθως γίνεται ακριβώς πριν την έξοδο του νήματος από το κρίσιμο τμήμα.

Τυπική χρήση wait- notify 1

```
synchronized method or block {  
    ...  
    try {  
        while (someTest) wait() ;  
    } catch (InterruptedException e) {}  
  
    ... // critical section  
    ... // modify someTest  
  
    notify(); // or notifyAll();  
    ...  
    // exit from synchronized method or block
```

Τυπική χρήση wait- notify 2

- Μπορεί το wait() και το notify() να βρίσκονται σε διαφορετικά synchronized blocks ή διαφορετικές synchronized μεθόδους στην ίδια κλάση, αλλά η εκτέλεση του κώδικα των blocks ή των μεθόδων πρέπει να παράγει το ίδιο αποτέλεσμα, δηλαδή τη δομή

```
while (someTest) wait() ;  
... // critical section  
... // modify someTest  
notify(); // or notifyAll();
```

Κριτική εποπτών Java

- Οι επόπτες στη Java είναι απλοί, κατανοητοί και εύχρηστοι.
- Έχει γίνει προσπάθεια αύξησης της διακριτότητας του αμοιβαίου αποκλεισμού του επόπτη, με τη χρήση `synchronized` τμημάτων.
- Η απλοποίηση των μεταβλητών συνθήκης διευκολύνει στο προγραμματισμό αλλά οδηγεί σε υπερβολικές αφυπνίσεις, αφού όλα τα νήματα περιμένουν στο ίδιο `wait set`.
- Το `wait()` διαθέτει `timeout` και διαχείριση διακοπών.
- Ο εναλλακτικός συνδυασμός είναι `Locks` και `Conditions` από το `java.util.concurrent`.

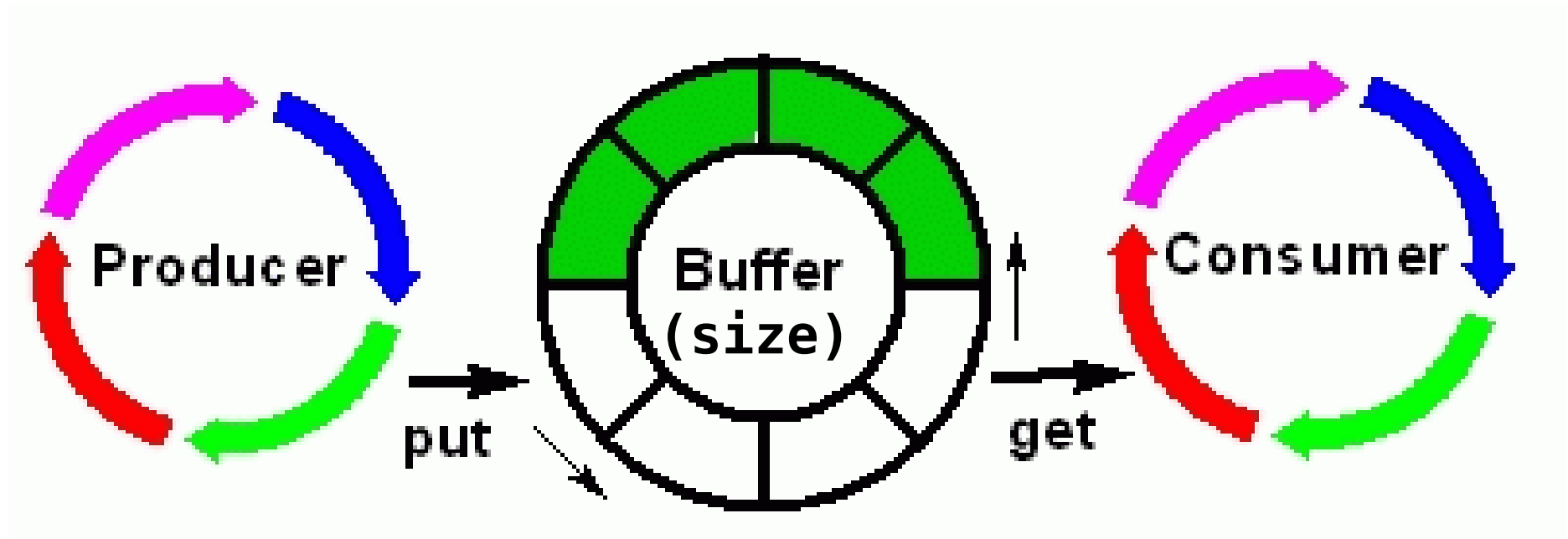


Παραγωγός καταναλωτής Producer Consumer ή Bounded Buffer

Συγχρονισμός και επικοινωνία μέσω μοιραζόμενου ενταμιευτή (buffer)

- Διεργασίες / νήματα που εκτελούνται ταυτόχρονα συγχρονίζονται και επικοινωνούν μέσω μοιραζόμενου ενταμιευτή (buffer).
- Συνήθως μοιραζόμενη ουρά: στο ένα άκρο εισάγονται δεδομένα (μηνύματα) και στο άλλο εξάγονται.
- Ο συγχρονισμός προκύπτει από την πιθανή αναμονή για ανάγνωση σε άδειο ενταμιευτή ή εγγραφή σε γεμάτο ενταμιευτή.
- Πολλές εφαρμογές: επικοινωνία στο διαδίκτυο, ουρές μηνυμάτων, κανάλια συζητήσεων, ρεύματα δεδομένων, επικοινωνία μέσω αρχείων..

Παραγωγός καταναλωτής Producer Consumer ή Bounded Buffer



Ο Παραγωγός (Producer) γράφει δεδομένα (put) σε ένα μοιραζόμενο ενταμιευτή (buffer)-κυκλική ουρά μεγέθους size, και ο Καταναλωτής (Consumer) τα διαβάει (get).

Παραγωγός καταναλωτής 2

Παραλλαγές:

- Μέγεθος ενταμιευτή (κυκλική ουρά)
size θέσεις, 1 θέση, άπειρες θέσεις.
- Αριθμός παραγωγών - καταναλωτών
SPSC, SPMC, MPSC, MPMC

Παραδοχές:

- Τα δεδομένα διαβάζονται μόνο μια φορά.
- Τα δεδομένα διαβάζονται με τη σειρά που έχουν γραφεί.
- Όλα τα δεδομένα που γράφονται τελικά διαβάζονται.

Παραγωγός καταναλωτής 3

Επίλυση δύο ζητημάτων

- Διαχείριση ταυτοχρονισμού στην προσπάθεια του Ενταμιευτή από παραγωγούς και καταναλωτές. Ταυτόχρονη προσπάθεια σε κυκλική ουρά.
- Συγχρονισμός σε συνθήκη παραγωγών και καταναλωτών. Ο συγχρονισμός εξαρτάται από τον ρυθμό εισαγωγής - εξαγωγής των δεδομένων, και από το μέγεθος του Ενταμιευτή.

Παραγωγός καταναλωτής

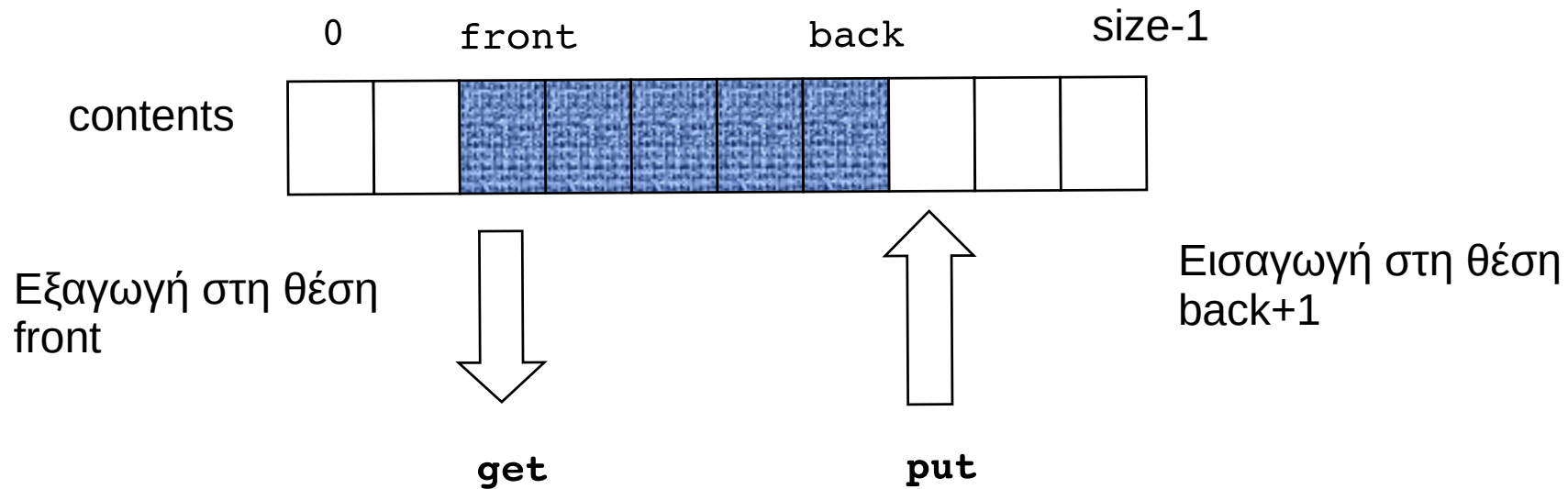
Αμοιβαίος αποκλεισμός

- Όταν ο Παραγωγός γράφει σε μια θέση του Ενταμιευτή, ο Καταναλωτής δεν πρέπει να διαβάζει την ίδια θέση.
- Η ενημέρωση του πλήθους των στοιχείων για διάβασμα (ή γράψιμο) στον Ενταμιευτή αποτελεί κρίσιμο τμήμα και πρέπει να επιβληθεί αμοιβαίος αποκλεισμός.

Παραγωγός καταναλωτής Συγχρονισμός σε συνθήκες

- Αν ο Ενταμιευτής είναι πλήρης, ο Παραγωγός δεν πρέπει να συνεχίσει να γράφει στοιχεία αλλά να αναμείνει εξαγωγή στοιχείων από τον Καταναλωτή.
- Αν Ενταμιευτής είναι κενός, ο Καταναλωτής δεν πρέπει να συνεχίσει να διαβάζει στοιχεία αλλά να αναμείνει εισαγωγή στοιχείων από τον Παραγωγό.

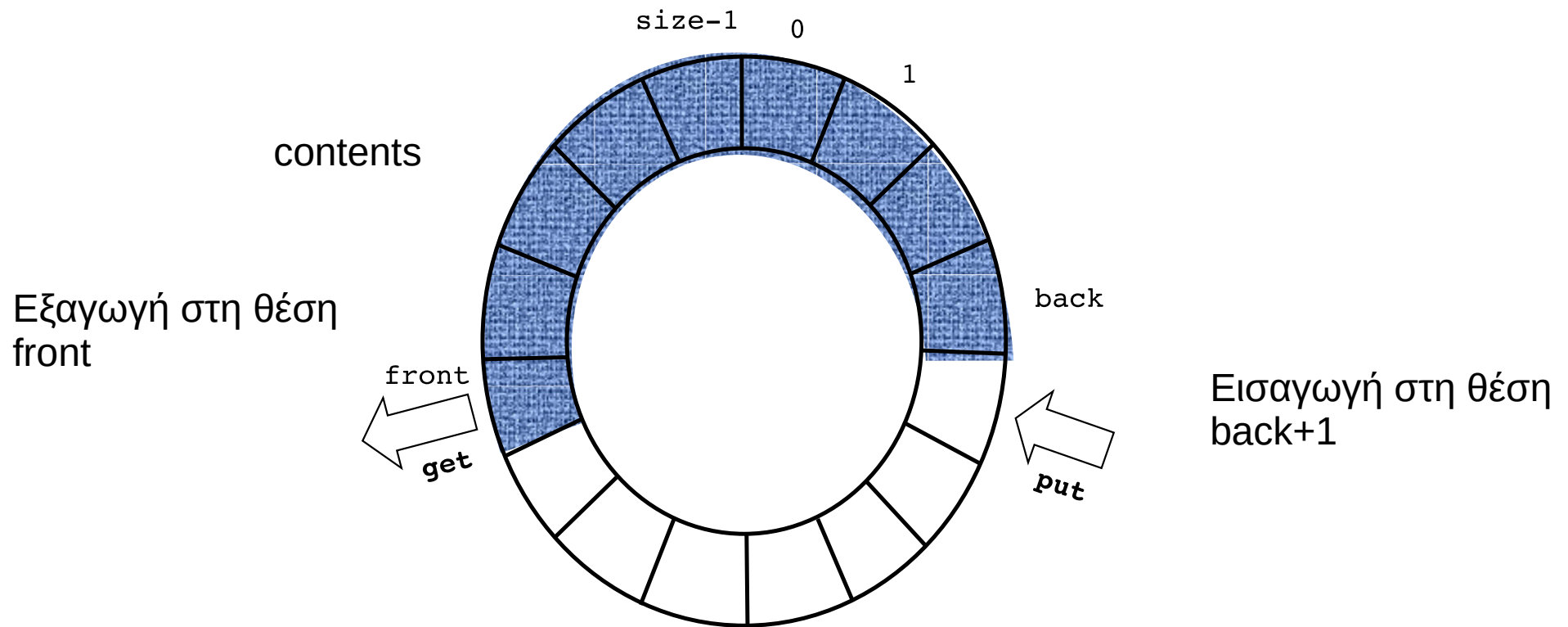
Ενταμιευτής: Κυκλική ουρά 1



Αρχικές τιμές: $back = size - 1$, $front = 0$, $counter = 0$.
Οι δείκτες $back$, $front$ αυξάνονται με χρήση $\% size$:

```
back = (back + 1) % size; counter++; put  
get; front = (front + 1) % size; counter--;
```

Ενταμιευτής: Κυκλική ουρά 2



`bufferEmpty:` `counter == 0`
`bufferFull:` `counter == size`

Γενικά: $| (\text{front} - \text{back}) \% \text{size} | == \text{counter} + 1$

Παραγωγός καταναλωτής: περίγραμμα 1

Προσοχή: μη-λύση!

```
//shared data
```

```
size,  
front = 0,  
back = size - 1,  
counter = 0,  
contents[size]; // buffer
```

```
main{  
    for (int i=0; i < number_of_Producers; i++)  
        Producer[i];  
  
    for (int j=0; j < number_of_Consumers; j++)  
        Consumer[j];  
}
```

Παραγωγός καταναλωτής: περίγραμμα 2

Προσοχή: μη-λύση!

```
Producer thread {
```

```
    for(int i = 0; i < number_of_items; i++){  
        produce(item);  
        put(item);  
        ... do something else for some time  
    }
```

```
}
```

```
put (data) {
```

```
    while (counter == size) {}                // busy wait  
    back = (back + 1) % size;  
    contents[back] = data;  
    counter++ ;
```

```
}
```

Παραγωγός καταναλωτής: περίγραμμα 3

Προσοχή: μη- λύση!

```
Consumer thread {
```

```
    while (true){  
        ... do something else for some time  
        value = get();  
        consume(value);  
    }
```

Μπορεί να θέσουμε μια μοιραζόμενη μεταβλητή για έλεγχο τερματισμού

```
}
```

```
data get() {
```

```
    while (counter == 0) {}           //busy wait  
    data = contents[front];  
    front = (front + 1) % size ;  
    counter--;  
    return data;  
}
```


Παραγωγός καταναλωτής: περίγραμμα 1

Προσοχή: υπο-βέλτιστη λύση!

```
//shared data
```

```
Lock_type l  
size,  
front = 0,  
back = size - 1,  
counter = 0,  
contents[size]; // buffer
```

```
main{  
    for (int i=0; i < number_of_Producers; i++)  
        Producer[i];  
  
    for (int j=0; j < number_of_Consumers; j++)  
        Consumer[j];  
}
```

Παραγωγός καταναλωτής: περίγραμμα 2

Προσοχή: υπο-βέλτιστη λύση!

Producer thread {

```
for(int i = 0; i < number_of_items; i++){  
    produce(item);  
    put(item);  
    ... do something else for some time  
}
```

```
}  
    put (data) {  
        while (true) {                                // busy wait  
            Lock(l)  
            if (counter < size) {  
                back = (back + 1) % size;  
                contents[back] = data;  
                counter++;  
                Unlock(l);  
                break;  
            }  
            Unlock(l);  
        }  
    }  
}
```

Παραγωγός καταναλωτής: περίγραμμα 3

Προσοχή: υπο-βέλτιστη λύση!

```
Consumer thread {  
    while (true){  
        ... do something else for some time  
        value = get();  
        consume(value);  
    }  
}  
  
data get() {  
    while (true) {                                //busy wait  
        Lock(l);  
        if (counter > 0) {  
            data = contents[front];  
            front = (front + 1) % size;  
            counter--;  
            Unlock(l);  
            return data;  
        }  
        Unlock(l);  
    }  
}
```

Μπορεί να θέσουμε μια μοιραζόμενη μεταβλητή για έλεγχο τερματισμού

Παραγωγός καταναλωτής

Σηματοφόροι: συγχρονισμός σε συνθήκη

```
//shared  
Semaphore itemsToGet(0);  
Semaphore itemsToPut(size);  
...
```

```
put(data) {
```

```
    P(itemsToPut)
```

```
    ...
```

```
    V(itemsToGet)
```

```
}
```

```
data get() {
```

```
    ...
```

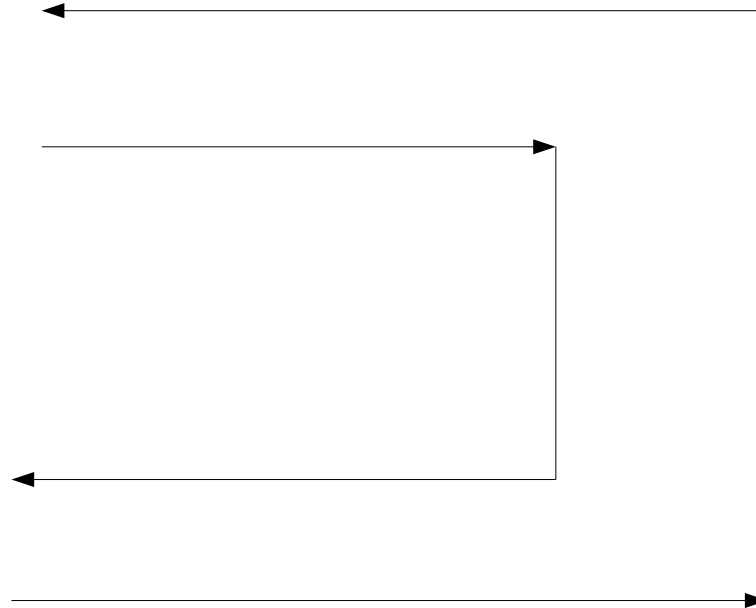
```
    P(itemsToGet)
```

```
    ...
```

```
    V(itemsToPut)
```

```
    ...
```

```
}
```



Αμοιβαίος αποκλεισμός

Σηματοφόροι: αμοιβαίος αποκλεισμός

```
//shared
Semaphore bufferMutex(1)

...
put(data){
    ...
    P(bufferMutex);
    back = (back + 1) % size;
    contents[back] = data;
    V(bufferMutex);
    ...
}

data get(){
    ...
    P(bufferMutex);
    data = contents[front];
    front = (front + 1) % size;
    V(bufferMutex);
    ...
    return data;
}
```

Αμοιβαίος αποκλεισμός ολόκληρου buffer.
Χρήσιμος όταν έχουμε πολλούς παραγωγούς ή και πολλούς καταναλωτές.

Παραγωγός καταναλωτής

Σηματοφόροι 1

```
//shared data
```

```
size,  
front = 0,  
back = size - 1,  
contents[size],  
Semaphore bufferMutex(1),  
Semaphore itemsToGet(0),  
Semaphore itemsToPut(size);
```

```
main{  
    for (int i=0; i < number_of_Producers; i++)  
        Producer[i];  
  
    for (int j=0; j < number_of_Consumers; j++)  
        Consumer[j];  
}
```

Παραγωγός καταναλωτής

Σηματοφόροι 2

Producer thread {

```
    for(int i = 0; i < number_of_items; i++){  
        produce(item);  
        put(item);  
        ... do something else for some time  
    }  
}
```

put(data) {

```
    P(itemsToPut);  
    P(bufferMutex);  
    back = (back + 1) % size;  
    contents[back] = data;  
    V(bufferMutex);  
    V(itemsToGet);  
  
}
```

Παραγωγός καταναλωτής

Σηματοφόροι 3

```
consumer thread {  
  
    while (true) {  
        ... do something else for some time  
        value = get();  
        consume(value);  
    }  
}
```

```
data get() {  
  
    P(itemsToGet);  
    P(bufferMutex);  
    data = contents[front];  
    front = (front + 1) % size;  
    V(bufferMutex);  
    V(itemsToPut);  
    return data;  
}
```


Παρατήρηση

- Θα μπορούσαμε να γράψουμε τον παρακάτω κώδικα; Ναι ή όχι και γιατί;

```
put(data) {  
  
    P(bufferMutex);  
    P(itemsToPut);  
    back = (back + 1) % size;  
    contents[back] = data;  
    V(itemsToGet);  
    V(bufferMutex);  
}
```

```
data get() {  
  
    P(bufferMutex);  
    P(itemsToGet);  
    data = contents[front];  
    front = (front + 1) % size;  
    V(itemsToPut);  
    V(bufferMutex);  
    return data;  
}
```

Παραγωγός καταναλωτής

Κλείδωμα και μεταβλητές συνθήκης

Συγχρονισμός σε συνθήκη

```
// shared
Lock_type l
Condition_type bufferEmpty
Condition_type bufferFull
counter = 0
...
```

```
put(data) :
    lock(l)
    while (counter == size) await(l, bufferFull)
    ...
    signal (bufferEmpty)
    unlock(l)

get() :
    lock(l)
    while (counter == 0) await(l, bufferEmpty)
    ...
    signal (bufferFull)
    unlock(l)
```

The diagram illustrates the synchronization between the `put` and `get` functions. It shows the flow of execution and how the state of the buffer is updated and signaled.

- put function:** Acquires the lock `l`. If the buffer is full (`counter == size`), it waits on the `bufferFull` condition variable. Once the buffer is not full, it signals the `bufferEmpty` condition variable and unlocks the lock.
- get function:** Acquires the lock `l`. If the buffer is empty (`counter == 0`), it waits on the `bufferEmpty` condition variable. Once the buffer is not empty, it signals the `bufferFull` condition variable and unlocks the lock.

Arrows in the diagram indicate the signaling flow: from `signal (bufferEmpty)` in `put` to the `await(l, bufferEmpty)` in `get`, and from `signal (bufferFull)` in `get` to the `await(l, bufferFull)` in `put`.



Παραγωγός καταναλωτής Κλείδωμα και μεταβλητές συνθήκης Αμοιβαίος αποκλεισμός

- Η προτεινόμενη λύση του συγχρονισμού σε συνθήκη προσφέρει λύση αμοιβαίου αποκλεισμού ολόκληρου του buffer.
- Κάθε στιγμή μόνο ένα νήμα μπορεί να κατέχει το κλείδωμα I.
- Τα υπόλοιπα είτε αναμένουν στις μεταβλητές συνθήκης είτε βρίσκονται έξω από το κλείδωμα.

Παραγωγός καταναλωτής

Κλείδωμα και μεταβλητές συνθήκης 1

```
// shared
```

```
Lock_type l  
Condition_type bufferEmpty  
Condition_type bufferFull  
size  
front = 0  
back = size - 1  
counter = 0  
contents[size]
```

```
main :
```

```
    for (int i=0; i < number_of_Producers; i++)  
        Producer[i]  
  
    for (int j=0; j < number_of_Consumers; j++)  
        Consumer[j]
```

Παραγωγός καταναλωτής

Κλείδωμα και μεταβλητές συνθήκης 2

Producer thread :

```
for(int i = 0; i < number_of_items; i++)  
    produce(item)  
    put(item)  
    ... do something else for some time
```

put(data) :

```
lock(l)  
while (counter == size) await(l, bufferFull)  
back = (back + 1) % size  
contents[back] = data  
counter++  
signal (bufferEmpty)  
unlock(l)
```

Παραγωγός καταναλωτής

Κλείδωμα και μεταβλητές συνθήκης 3

consumer thread :

```
while (true)
    ... do something else for some time
    value = get()
    consume(value)
```

get() :

```
lock(l)
while (counter == 0) await(l, bufferEmpty)
data = contents[front]
front = (front + 1) % size
counter--
signal (bufferFull)
unlock(l)
return data
```

Γενικές παρατηρήσεις: σύγκριση λύσεων

- Μελετήστε τους κώδικες που σας έχουν δοθεί για τη λύση του προβλήματος παραγωγού καταναλωτή και συγκρίνετε τις υλοποιήσεις με τον ψευδοκώδικα.
- Παρατηρήστε τη λύση με επόπτη σε Java και συγκρίνετέ την με τις υπόλοιπες λύσεις.
- Είναι προτιμότερες δύο μεταβλητές συνθήκης; Ναι, όχι, γιατί;
- Τι περιμένουμε από τη λύση με επόπτη σε Java για μεγάλο αριθμό επαναλήψεων;



Παραγωγός Καταναλωτής

- Code/ProdCons